

Programação Orientada a Objetos

Introdução à POO

Prof. Chauã Queirolo

1

Sumário

Referências bibliográficas

2

Por que estudar POO?

Problemas comuns no desenvolvimento:

- Sistemas cada vez maiores
- Equipes de desenvolvimento maiores
- Manutenção frequente
- Necessidade de reutilização de código

Paradigmas de Programação

- Forma de pensar e organizar um programa quanto a:
 - Estrutura
 - Execução
- Principais paradigmas:
 - **Imperativo:** procedural, modular, orientado a objetos
 - **Declarativo:** funcionalista, lógico

Programação Procedural

Organização

- Dados
- Procedimentos (funções)

Exemplo

- lerDados()
- processarDados()
- gerarRelatorio()

Estrutura típica

dados → funções → execução sequencial

Programação Procedural

Características

- Fluxo sequencial
- Forte dependência entre funções
- Dados frequentemente globais

Sistemas grandes

- Difícil manutenção
- Forte acoplamento entre funções
- Alterações podem quebrar outras partes do sistema
- Código pouco reutilizável

Funciona bem em sistemas pequenos

Programação Orientada a Objetos

- Na Programação Orientada a Objetos a organização muda
- Em vez de focar em funções, focamos em objetos

Objeto = **Dados** + **Comportamento**

Programação Orientada a Objetos

- Descreve o sistema com **elementos do mundo real**
- Considera que todos os componentes são **objetos**
- **Objetos** possuem sua **estrutura** e desempenham **ações**
- **Classificados** de acordo com suas **características**

- **Exemplo:** Java, C++, C#, Python, Dart, Kotlin, Ruby

Programação Orientada a Objetos

Vantagens

- Abstração
- Modularização
- Extensibilidade
- Reaproveitamento de código

9

Objetos

Representa uma **entidade** do **mundo real** dentro do programa

Cada objeto possui

- **Características**
- **Funções**

O universo é formado por objetos



10



Classes

- Os **objetos** são **classificados** de acordo com
 - **Características** semelhantes
 - **Funcionalidades** semelhantes

Classes são agrupamentos objetos semelhantes entre si

Classes

Pessoa
- nome - idade - peso - altura
+ andar() + conversar() + dormir() + dirigirCarro()

Atributos

- características, propriedades

Métodos

- funcionalidades, ações, procedimentos

Relacionamento

- Associação
- Agregação
- Composição
- Herança

13



Pilares da Orientação a Objetos

- Encapsulamento
- Herança
- Polimorfismo

14

Encapsulamento

- Todo objeto é responsável pelos seus atributos
- Esconder do mundo externo:
 - Estrutura interna dos objetos
 - Detalhes de implementação
- Interação através de uma interface pública

15

Herança

- Estabelece a relação **é um** entre duas classes
- Permite
 - **Abstração**
 - **Reaproveitamento** de código

16

Polimorfismo

- Um objeto pode assumir **diferentes formas**
- Tipos de polimorfismo:
 - **por inclusão** via herança
 - **paramétrico** tipos genéricos
 - **sobrescrita** redefinição de métodos
 - **sobrecarga** métodos com mesmo nome, parâmetros diferentes

Referências bibliográficas

- SINTES, Anthony. **Aprenda Programação Orientada a Objetos em 21 dias**. Pearson.